

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

MASTER TEST PLAN

Ingegneria del Software: Tecniche Avanzate – Master Test Plan

---

Mario Celzo	Matricola: NF22500193
Renato Mancino	Matricola: NF22500242
Alfonso Maria Ferrara	Matricola: NF22500210

Repository: `https://github.com/mariocelzo/biblioflow-app`

Anno Accademico 2025-2026

Indice

1 Introduzione **1**

2 Strategie di Testing **2**

2.1 Testing di unità 2

2.2 Testing di integrazione 2

2.3 Testing di sistema 2

2.4 Testing E2E per la WebApp 3

3 Criteri di Accettazione **4**

3.1 Criteri di Copertura dei Test 4

3.1.1 Funzionalità incluse nel Testing 4

3.1.2 Funzionalità escluse dal Testing 4

3.2 Criteri e Valori di Accettazione 4

CAPITOLO 1

Introduzione

Questo documento definisce le strategie pianificate per il testing di BiblioFlow. Il piano definisce i diversi livelli di test: test di unità, di integrazione, di sistema ed E2E, con l'obiettivo di garantire la qualità del software considerando anche le nuove funzionalità descritte nel documento Change Requests.

Verranno escluse dal perimetro di testing le integrazioni con servizi esterni che il progetto usa ma non realizza (il recapito effettivo delle email, il fornitore di accesso Google e gli strumenti di monitoraggio) e l'installazione della PWA.

CAPITOLO 2

Strategie di Testing

2.1 Testing di unità

La fase di testing di unità è finalizzata alla verifica dei singoli componenti di BiblioFlow, con l'intento di isolare e validare il comportamento dei moduli critici: servizio di dominio delle prenotazioni, autenticazione, automazioni e route handler. La metodologia adottata è di tipo white-box, focalizzata sui criteri di statement coverage e branch coverage misurati con Vitest. Le dipendenze esterne, cioè il client Prisma e la sessione di Auth.js, sono sostituite da mock, così che ogni test verifichi solo la logica del modulo.

Allo stesso livello appartengono i test dei componenti React coinvolti dalle Change Requests (mappa dei posti, griglia mobile, lista d'attesa, azioni dello staff), eseguiti in un DOM simulato con Testing Library e interrogati come li userebbe una persona: per ruolo e nome accessibile.

I test sono organizzati per livello: `tests/unit` (moduli e route con dipendenze simulate; componenti React in `tests/unit/componenti`), `tests/integration` e `tests/post-modifica` (PostgreSQL reale), `tests/pre-modifica` (caratterizzazione della versione di partenza) e `tests/e2e` (Playwright). Ogni file prende il nome del modulo o della funzionalità che verifica.

2.2 Testing di integrazione

La fase di testing di integrazione ha lo scopo di verificare la comunicazione tra i sottosistemi di BiblioFlow. L'attività si concentra sulla catena completa che va dal route handler al servizio di dominio fino a PostgreSQL, eseguita contro un database PostgreSQL reale e isolato (il servizio di test locale o quello effimero della pipeline). Le fixture condivise e la preparazione della suite E2E rifiutano un database il cui nome non dichiari di essere di test; i file che usano direttamente il client applicativo si affidano alla variabile `DATABASE_URL` dell'ambiente di test. Nello specifico verranno esaminati il vincolo anti-sovrapposizione, le transazioni concorrenti sullo stesso posto, il ciclo di vita della lista d'attesa e la promozione eseguita dalle automazioni, con la verifica di notifiche ed eventi prodotti.

2.3 Testing di sistema

Il testing di sistema verrà eseguito secondo un approccio black-box, usando come confine del sistema le API HTTP esposte dai route handler. La metodologia di riferimento per la progettazione dei casi di test è il Category Partition Method, che permette di individuare e definire sistematicamente le combinazioni di input. I casi progettati sulla versione di partenza ne fissano il comportamento prima di ogni modifica; la strategia dovrà poi coprire i cambiamenti apportati al sistema dalle Change Requests.

2.4 Testing E2E per la WebApp

Il testing End-to-End (E2E) ha il compito di validare la WebApp nel suo complesso, simulando percorsi di utilizzo reali in un browser per garantire la coerenza tra tutti i sottosistemi. I test si definiscono sui percorsi principali: l'accesso alle pagine riservate, la prenotazione guidata, il posto occupato con l'ingresso in lista d'attesa, la promozione dopo una cancellazione e l'uscita dalla coda. I criteri di successo richiedono che l'intero ciclo si svolga senza interruzioni sulla build di produzione. L'intera procedura è automatizzata tramite Playwright.

CAPITOLO 3

Criteri di Accettazione

3.1 Criteri di Copertura dei Test

3.1.1 Funzionalità incluse nel Testing

L'attività di testing coprirà le componenti architetturali impattate dalle modifiche (CR01 e CR02) e le funzionalità core necessarie al loro funzionamento. Nello specifico:

- **Prenotazione dei posti (CR01):** route handler delle prenotazioni, servizio di dominio, autenticazione e middleware, schema e vincoli del database.
- **Lista d'attesa e riassegnazione (CR02):** route della coda, automazioni periodiche, cancellazioni dello studente e dello staff, notifiche ed eventi.
- **Area staff:** route amministrative toccate dalle modifiche e controllo dei ruoli su tutte le route dello staff.
- **Interfaccia di prenotazione:** mappa, griglia mobile, messaggio di conflitto e gestione della propria coda.
- **Regressione:** accesso, prestiti, statistiche e notifiche esistenti, rieseguendo i casi della versione di partenza.

3.1.2 Funzionalità escluse dal Testing

Sono escluse dal perimetro di questo piano:

- **Servizi esterni:** il recapito delle email tramite il fornitore di posta e l'accesso con Google sono sostituiti da mock; resta verificato il comportamento di BiblioFlow quando il servizio risponde o fallisce.
- **Monitoraggio e analisi d'uso:** Sentry, Vercel Analytics e Speed Insights.
- **Installazione PWA:** il comportamento del service worker e della modalità offline, che dipende dal browser e non è toccato dalle Change Requests.

3.2 Criteri e Valori di Accettazione

Il successo della versione aggiornata di BiblioFlow deve garantire i seguenti criteri:

- **Copertura del Codice:** almeno il 90% di statement coverage e l'80% di branch coverage su `src/lib` e `src/app/api`, e il 100% sul middleware. Le soglie sono configurate in Vitest: `npm run test:coverage` fallisce se non vengono raggiunte.

- **Validazione Funzionale:** il sistema deve rispondere correttamente alle modifiche definite nelle Change Requests, in particolare:
  - **CA-01, accesso:** ogni operazione su prenotazioni e coda richiede la sessione; uno studente non vede né modifica risorse altrui e non accede all'area staff;
  - **CA-02, concorrenza:** con richieste simultanee sullo stesso posto e intervallo viene creata al più una prenotazione, senza record parziali;
  - **CA-03, lista d'attesa:** l'ordine è quello di arrivo e la stessa persona non può avere due richieste attive per lo stesso posto e intervallo;
  - **CA-04, riassegnazione:** quando il posto si libera il primo in coda viene promosso una sola volta, anche con esecuzioni concorrenti, e la promozione non confermata scade;
  - **CA-05, tracciabilità:** ingresso, uscita, promozione e scadenza producono eventi correlati, e ingresso, promozione e scadenza anche una notifica;
  - **CA-06, regressione:** le funzionalità esistenti non toccate dalle modifiche mantengono il loro comportamento.
- **Stabilità:** non devono essere presenti bug bloccanti nei processi di testing e la suite E2E deve risultare stabile su esecuzioni ripetute.
- **Regressione:** tutti i casi della versione di partenza classificati come invariati devono continuare a passare, a conferma che le modifiche non hanno introdotto regressioni nelle funzionalità esistenti.